



UNIVERSITÄT
LEIPZIG

4. Vorlesung: Regression & Nearest Neighbor Klassifikation

INTERACTIVE VISUAL DATA MINING

Übersicht

4. Regression und k-NN

4.1 Korrelation

4.2 Lineare Regression

4.3 Logistische Regression

4.4 Nearest Neighbor

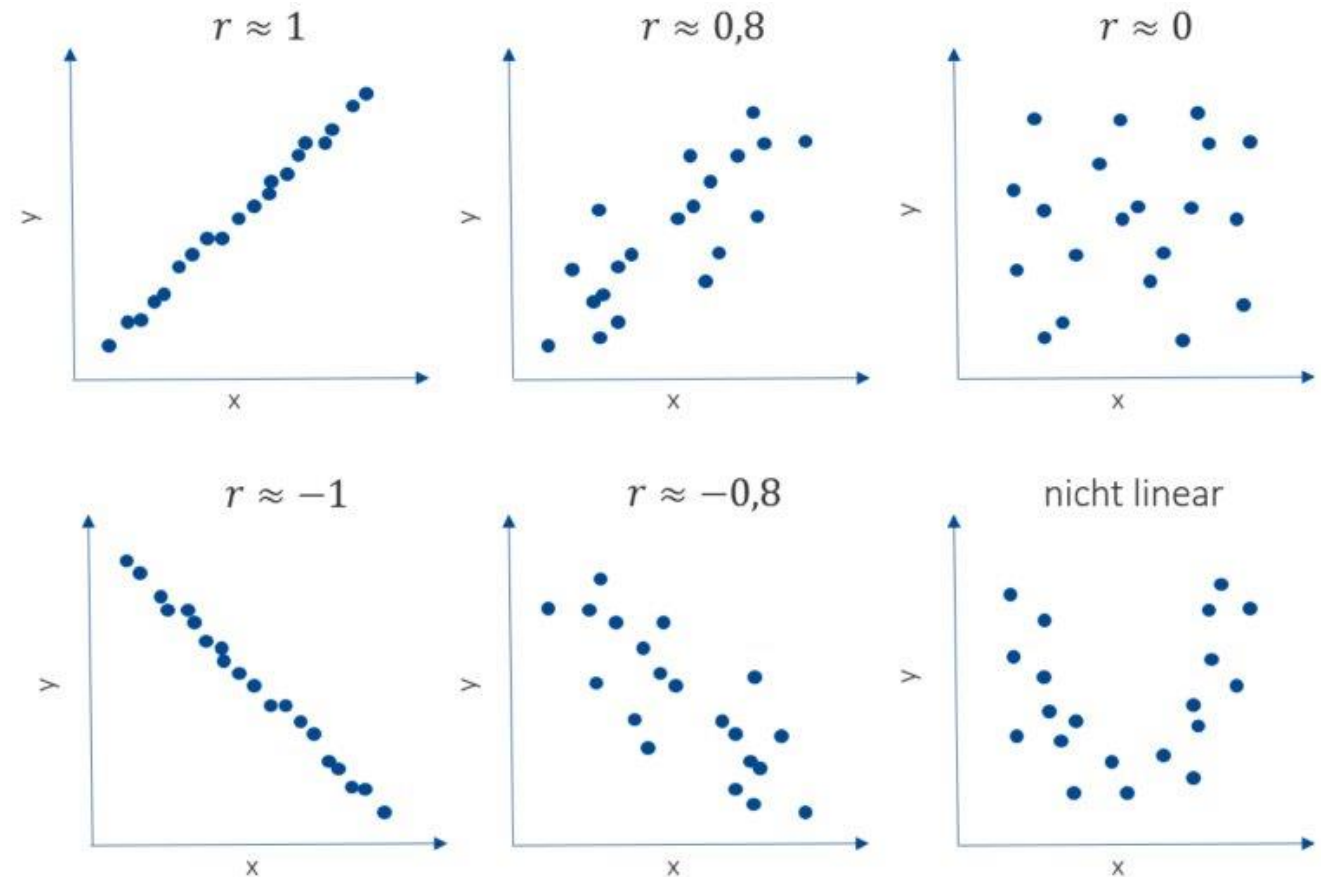
4.5 Nearest Neighbor Datenstrukturen

Lernziele:

- Die Grundidee von Korrelation und Regression verstehen
- Lineare Regression verstehen sowie die Bedeutung von Gewichten und Fehlern erklären können
- Logistische Regression als Verfahren zur Klassifikation verstehen und von linearer Regression unterscheiden können
- Das k-Nearest-Neighbor-Verfahren (k-NN) erklären und den Einfluss von Distanzmaßen auf das Ergebnis verstehen
- Effiziente Datenstrukturen für k-NN kennen und deren Grundprinzip erklären sowie einen k-d-Baum konstruieren können

Korrelation

- Mit Korrelations- und Regressionsanalyse werden Zusammenhänge zwischen zwei metrischen Variablen analysiert
- Korrelation beschreibt, ob und wie stark zwei Variablen miteinander zusammenhängen
 - Positive Korrelation: Steigt Variable A, steigt auch B
 - Negative Korrelation: Steigt A, sinkt B
 - Keine Korrelation: Kein erkennbarer linearer Zusammenhang



Korrelation

- Mit Korrelations- und Regressionsanalyse werden Zusammenhänge zwischen zwei metrischen Variablen analysiert
- Korrelation beschreibt, ob und wie stark zwei Variablen miteinander zusammenhängen
 - Positive Korrelation: Steigt Variable A, steigt auch B
 - Negative Korrelation: Steigt A, sinkt B
 - Keine Korrelation: Kein erkennbarer linearer Zusammenhang

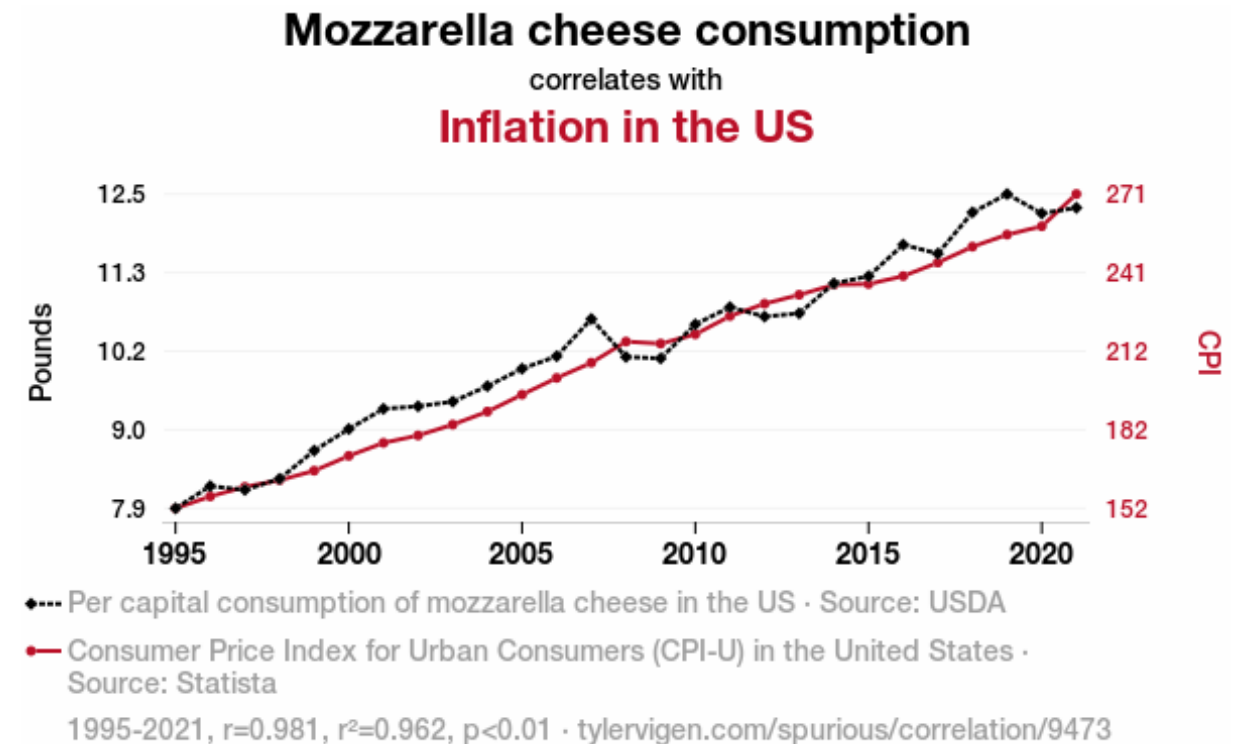
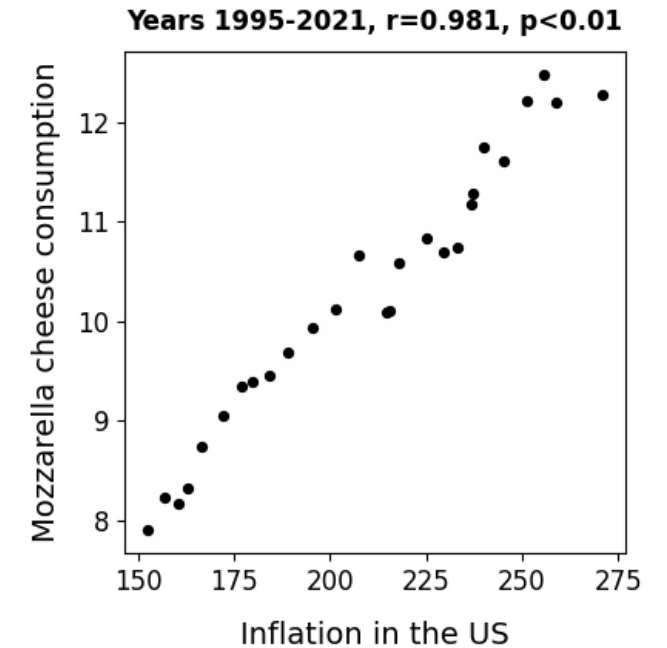
The diagram shows the formula for the Pearson correlation coefficient r with several explanatory arrows:

- An arrow points from the text " x_i sind die **einzelnen Werte** der einen **Variable**" to the x_i term in the numerator.
- An arrow points from the text " y_i sind die **einzelnen Werte** der anderen **Variable**" to the y_i term in the numerator.
- An arrow points from the text "Dabei ist r der **Pearson-Korrelationskoeffizient**," to the r on the left side of the equation.
- An arrow points from the text " $\bar{x}$ und $\bar{y}$ sind jeweils die **Mittelwerte** der beiden Variablen" to the $\bar{x}$ and $\bar{y}$ terms in the denominator.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Korrelation

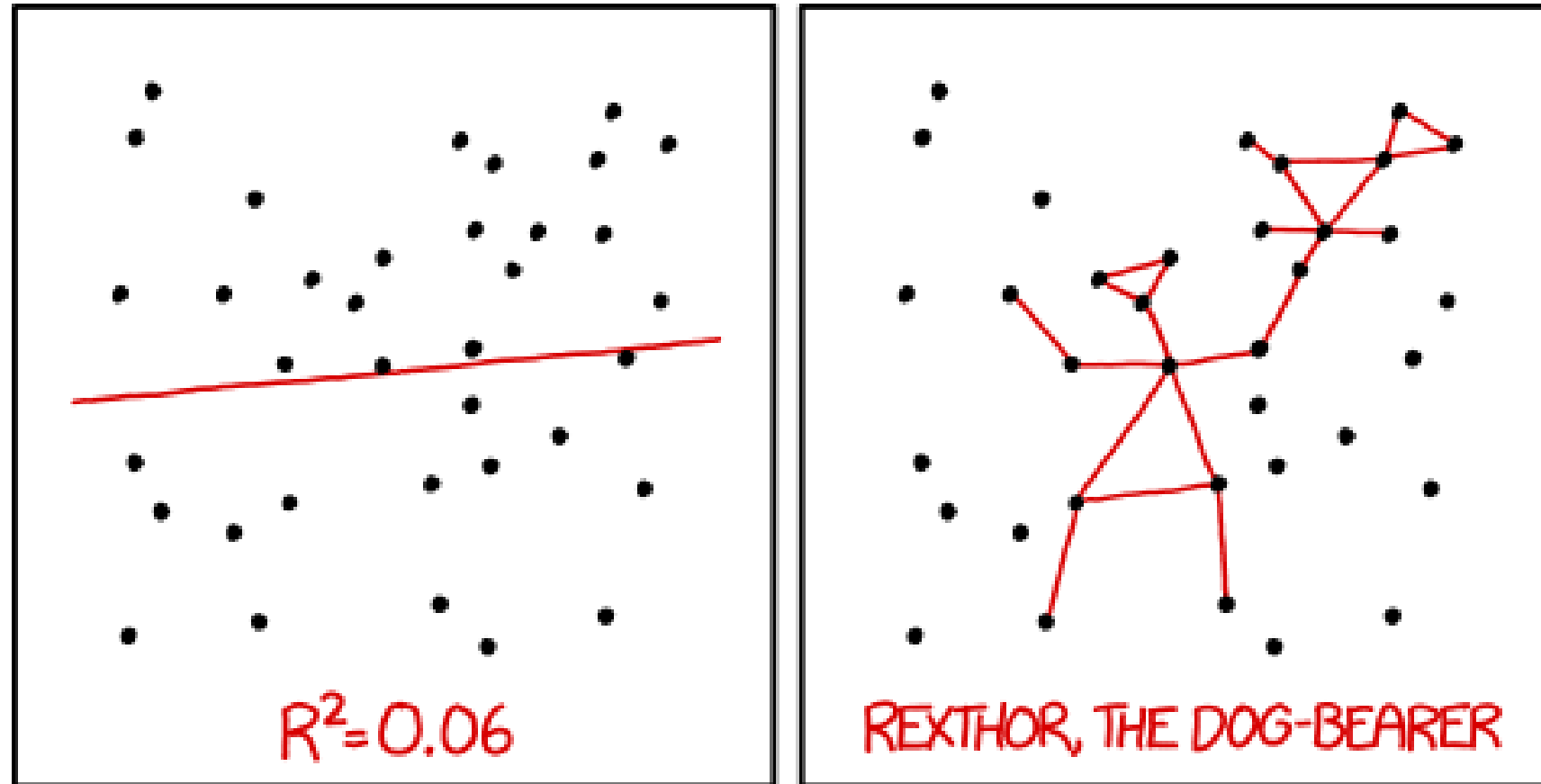
- Mit Korrelations- und Regressionsanalyse werden Zusammenhänge zwischen zwei metrischen Variablen analysiert
- Korrelation beschreibt, ob und wie stark zwei Variablen miteinander zusammenhängen
 - Positive Korrelation: Steigt Variable A, steigt auch B
 - Negative Korrelation: Steigt A, sinkt B
 - Keine Korrelation: Kein erkennbarer linearer Zusammenhang



Korrelation

- Korrelation $\neq$ Kausalität
 - Nur weil zwei Variablen zusammenhängen, bedeutet das nicht, dass eine Variable die andere verursacht
- Mögliche Gründe für Zusammenhänge:
 - Zufall
 - Drittvariable (Confounder) beeinflusst beide Variablen
 - Umgekehrte Kausalität (Y beeinflusst X statt umgekehrt)
- Konsequenz
 - Korrelation zeigt nur Zusammenhänge, keine Ursache-Wirkung
 - Kausale Aussagen erfordern zusätzliche Annahmen oder geeignete Studiendesigns
- Regression modelliert Zusammenhänge und ermöglicht Vorhersagen, aber beweist ebenfalls keine Kausalität

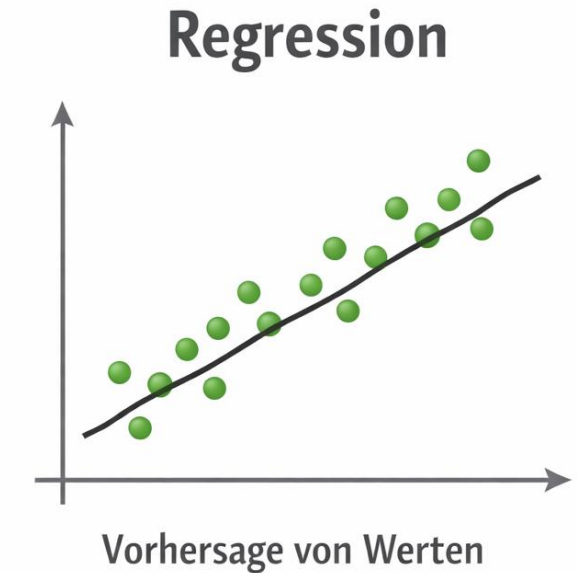
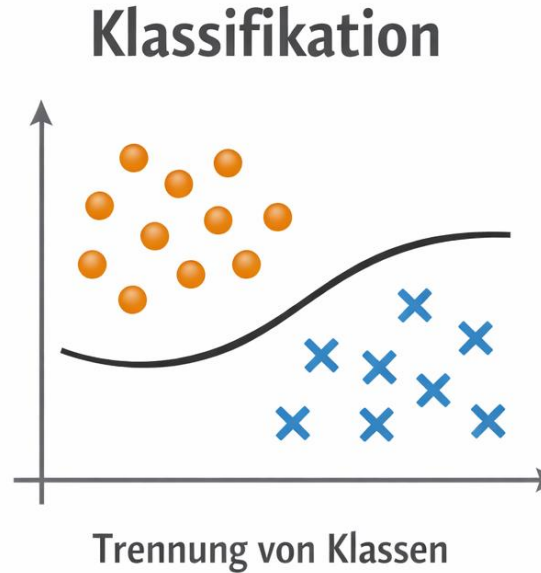
Regression



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

Regression

- Statistisches Verfahren, um Beziehungen zwischen einer abhängigen und einer oder mehreren unabhängigen Variablen zu modellieren
- Ziele der Regression:
 - Vorhersage eines numerischen Ergebnisses anhand von Input-Variablen
 - Analyse, wie stark Variablen miteinander zusammenhängen
 - Unterstützung von Entscheidungen durch Prognosen
- Regression kann **linear** oder **nicht-linear** sein, je nachdem, wie die Beziehung zwischen den Variablen aussieht.



Lineare Regression

Einfache Lineare Regression

- Ziel: Vorhersage eines numerischen Werts Y auf Basis einer einzelnen numerischen Einflussvariable X .
- Es wird angenommen, dass zwischen X und Y ein annähernd linearer Zusammenhang besteht.
- Gesucht: β_0 und β_1 die Parameter des Modells.
 - Entsprechen dem Achsenabschnitt und der Steigung einer Geraden.

$$Y = \beta_0 + \beta_1 X + \epsilon$$

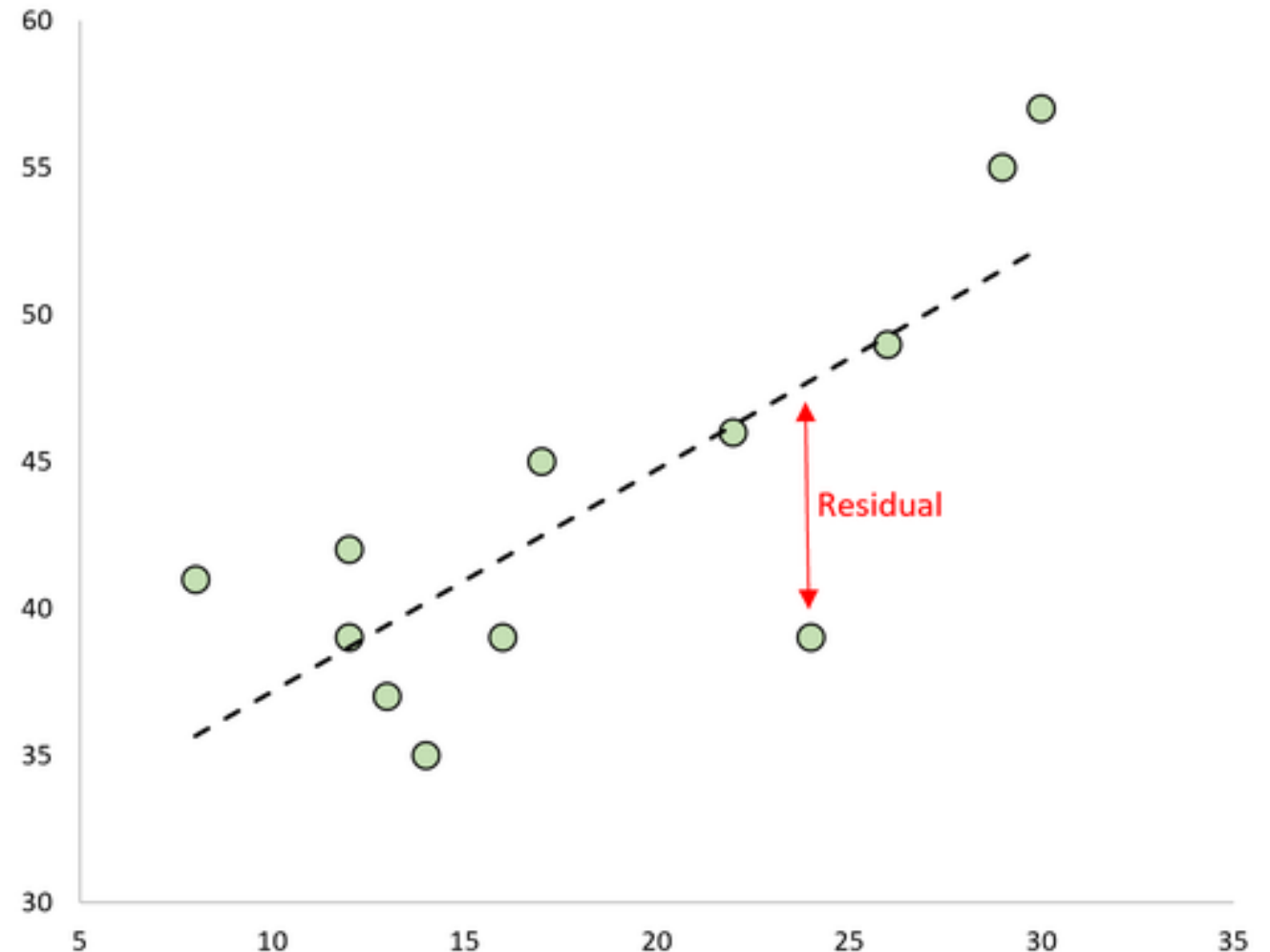
Zusammenhang mit Korrelation

- Korrelation misst lineare Beziehungen zwischen Variablen
- Hohe Korrelation → gute Grundlage für lineare Regression
- Niedrige/keine Korrelation → geringe Vorhersagekraft

Lineare Regression

Residuen (Fehler)

- Differenz zwischen tatsächlichem Wert und vorhergesagtem Wert
- Zeigt, wie gut das Modell jede Instanz vorhersagt
- Ziel: Residuen möglichst klein halten (Least Squares minimiert die Summe der quadrierten Residuen)
- Grafisch: Abstand der Punkte zur Regressionslinie



Lineare Regression

$$\hat{\beta}_0, \hat{\beta}_1 = \arg \min_{\beta_0, \beta_1} \left[\sum_{i=1}^n \left(y^{(i)} - \left(\beta_0 + \beta_1 x^{(i)} \right) \right)^2 \right]$$

- Die Gewichte/Parameter werden so gewählt, dass der Vorhersagefehler minimal wird
 - In der Praxis wird meist die Summe der quadrierten Fehler minimiert → Least Squares (Methode der kleinsten Quadrate)
 - Die Gewichte, basieren auf den Trainingsdaten und werden für die Vorhersage der Instanz unbekannter Daten genutzt
- n Instanzen
 - $y^{(i)}$ Wert von Attribut Y von Instanz i
 - $x^{(i)}$ Wert von Attribut X von Instanz i

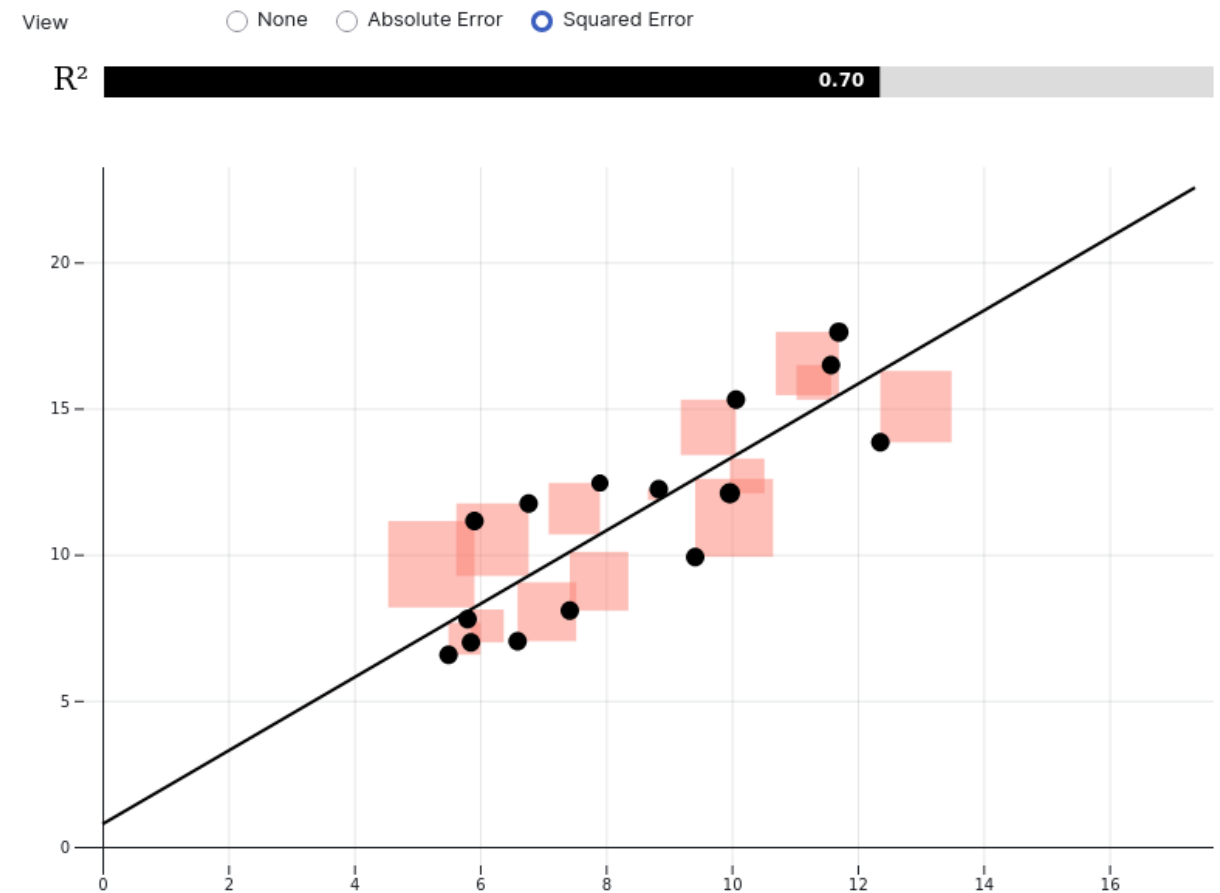
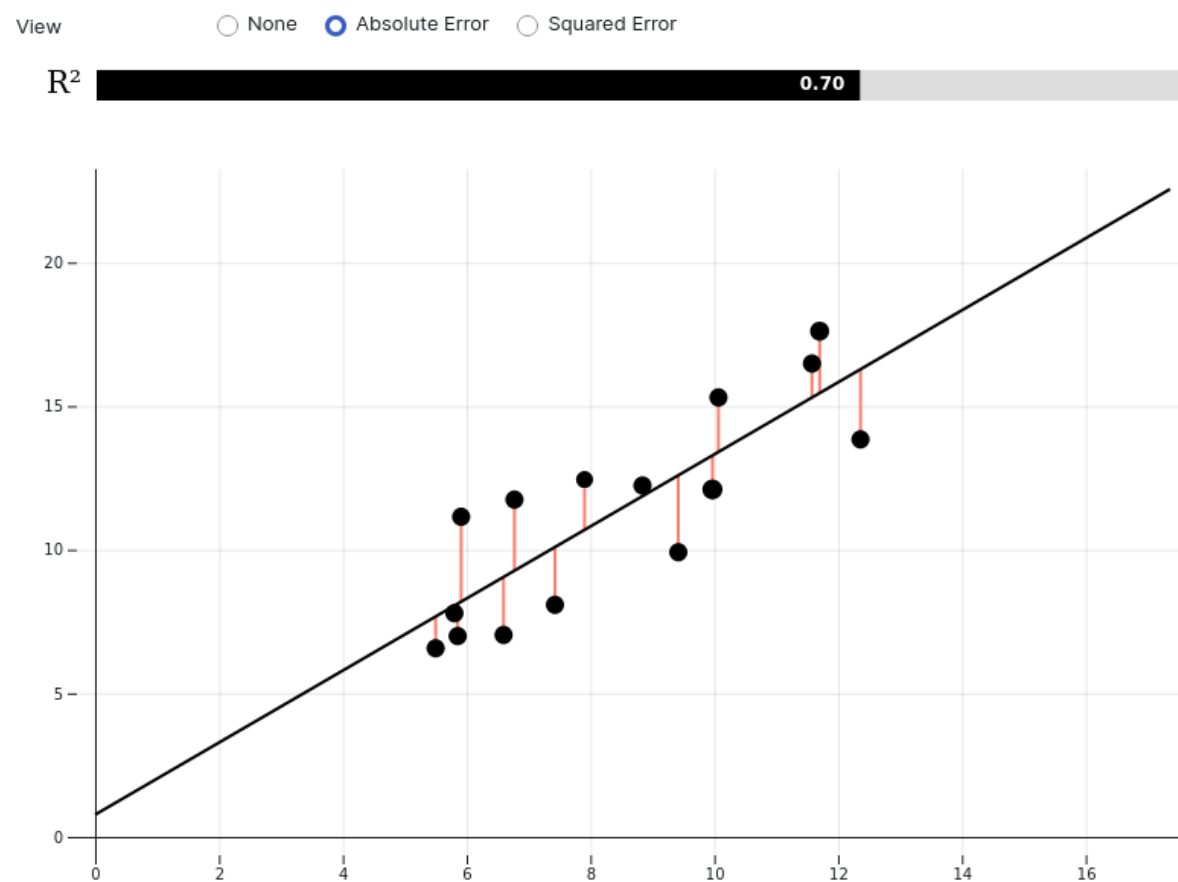
Lineare Regression

Störgrößen (Noise)

- In realen Daten gibt es unerklärbare Abweichungen
- Ursachen:
 - Messfehler
 - Nicht berücksichtigte Einflussgrößen
 - Zufällige Schwankungen
 - Interaktionen und nichtlineare Effekte
- Konsequenz: Auch das beste lineare Modell kann nicht alle Werte perfekt vorhersagen
- Zusammenhang:
 $\text{Residuen} = \text{Modellfehler} + \text{Störfaktor}$

Lineare Regression

- Demo
 - <https://observablehq.com/@yizhe-ang/interactive-visualization-of-linear-regression>



Lineare Regression

Multiple Lineare Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

- Ziel: Vorhersage eines numerischen Werts Y auf Basis von mehreren numerischen Werten
- Annahme: Das Ergebnis lässt sich als lineare Kombination der Attribute darstellen
- Die Anzahl der Trainingsinstanzen n sollte deutlich größer als die Anzahl der Attribute p sein
- Berechnung wieder über Least Squares

$$\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n \left(y^{(i)} - \left(\beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)} \right) \right)^2$$

Lineare Regression

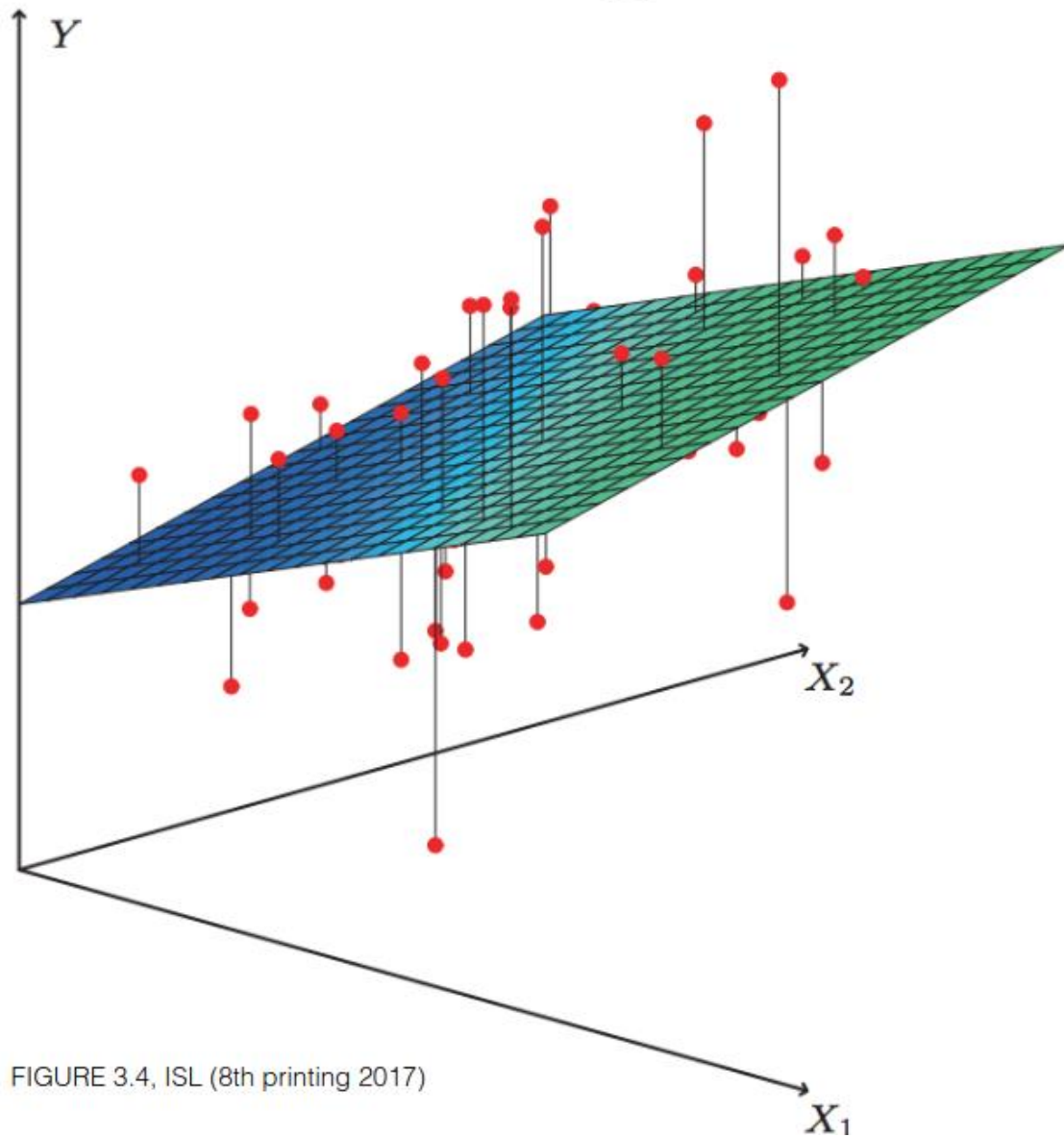
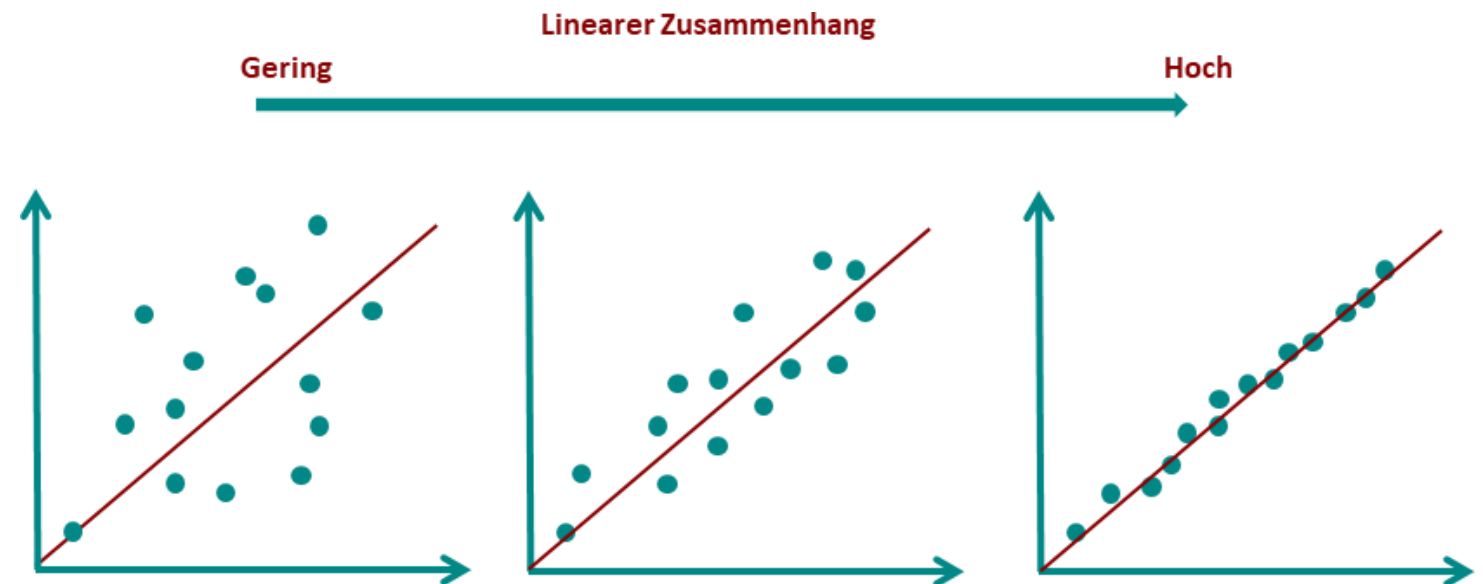


FIGURE 3.4, ISL (8th printing 2017)

Linear Regression

- Eigenschaften:
 - Einfache Methode und leicht zu interpretieren
 - Wird oft in statistischen Analysen genutzt
 - Nur für Daten mit linearen Abhängigkeiten geeignet
 - Wenn die Daten nicht linear abhängig sind, funktioniert das Verfahren nicht!



Logistische Regression

- Ziel: Klassifikation (z. B. 0/1)

$$Pr(Y = 1|X) = \beta_0 + \beta_1 X$$

- Problem bei Linearer Regression: Werte können < 0 oder > 1 sein. Sind also keine Wahrscheinlichkeiten
- Lösung: Bau ein lineares Modell auf einer transformierten Zielvariable
- Wahrscheinlichkeiten werden mit logit-Transformation in Bereich $[-\infty, \infty]$ abgebildet

$$\ln \left(\frac{P(Y = 1 | X)}{1 - P(Y = 1 | X)} \right) = \beta_0 + \beta_1 X$$

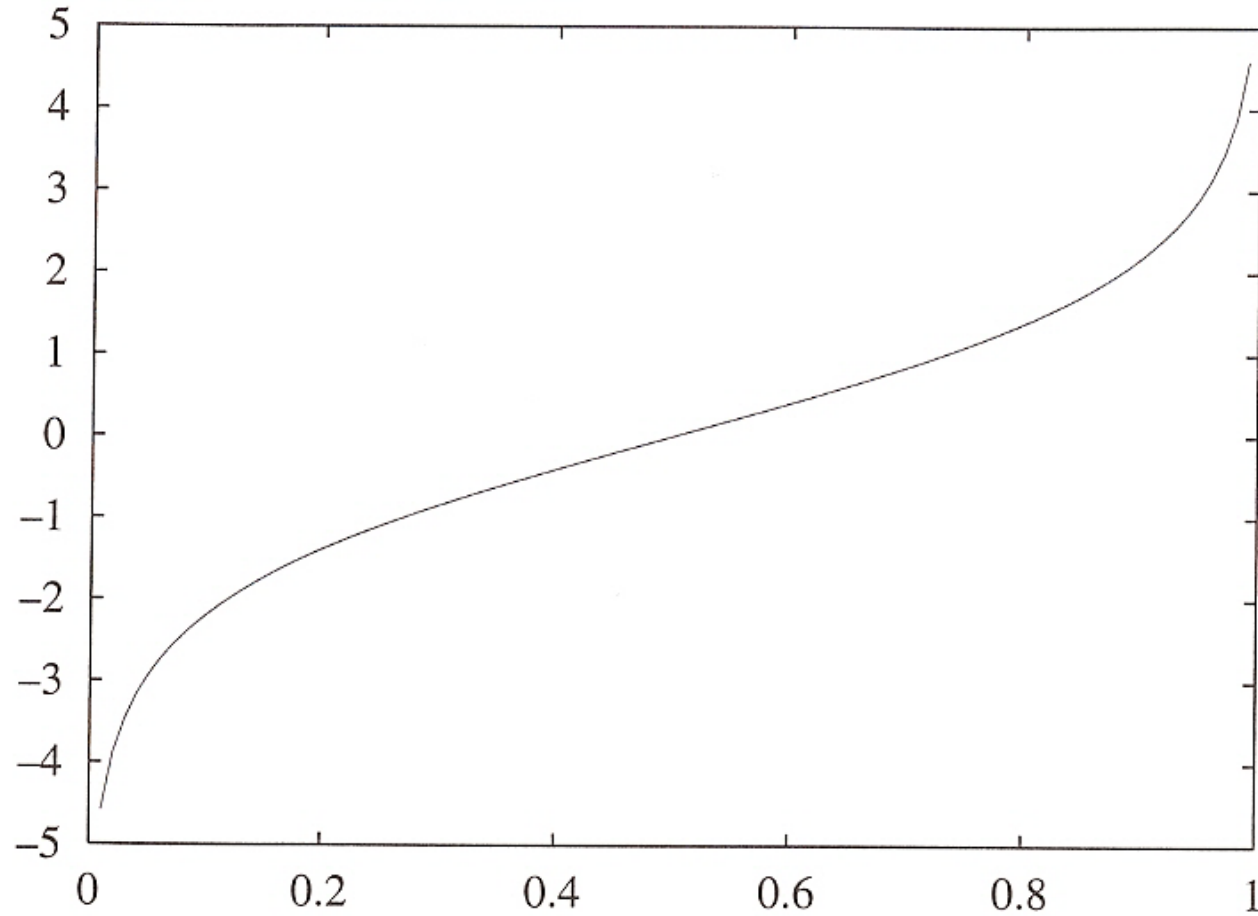
- Die inverse Funktion ist eine Sigmoidfunktion die Werte auf $[0, 1]$ begrenzt (S-förmig)

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

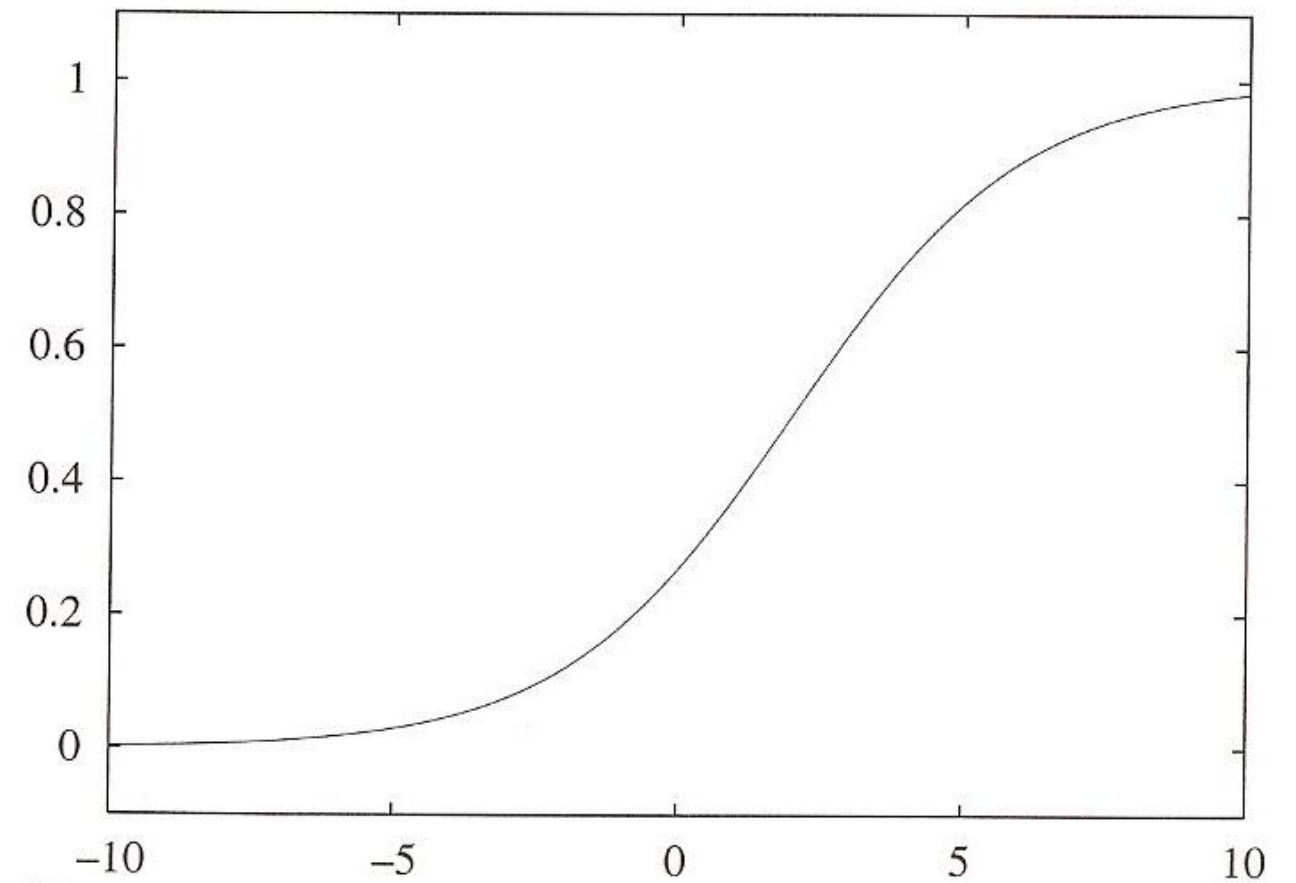
- Die transformierte Variable wird wie zuvor durch eine lineare Funktion approximiert
- Logistisches Regressions Modell:

$$Pr(Y = 1 | X) = \sigma(\beta_0 + \beta_1 X) = \frac{1}{1+e^{-(\beta_0+\beta_1 X)}}$$

Logistische Regression

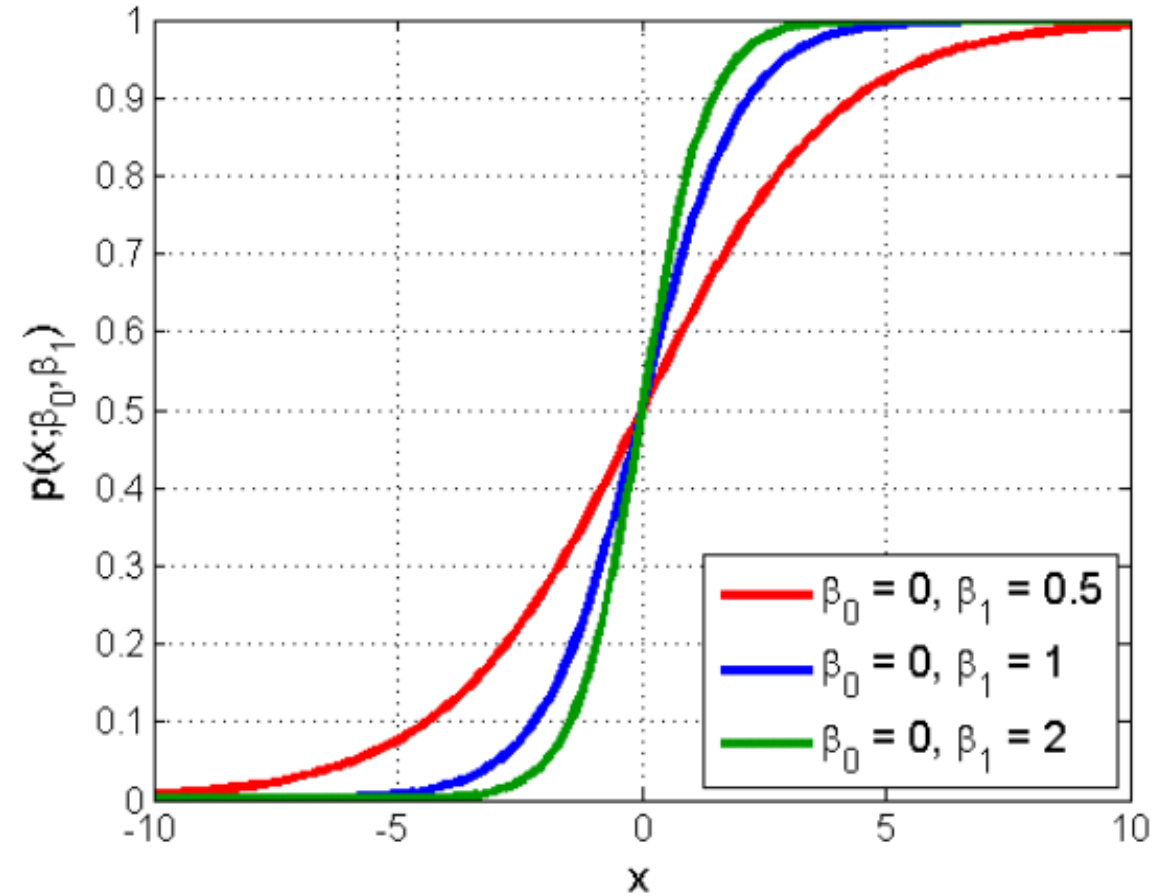
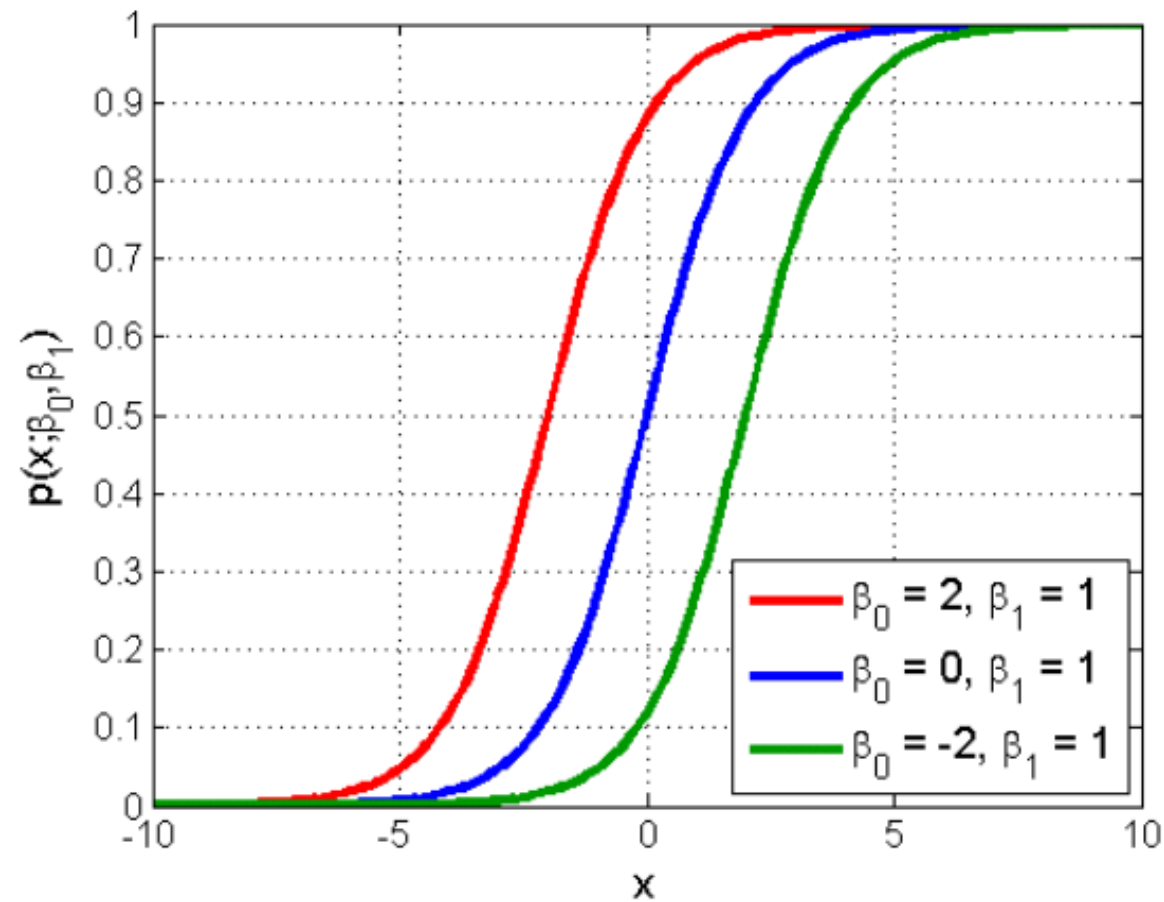


Logit-Transformation



Logistische Regression Funktion

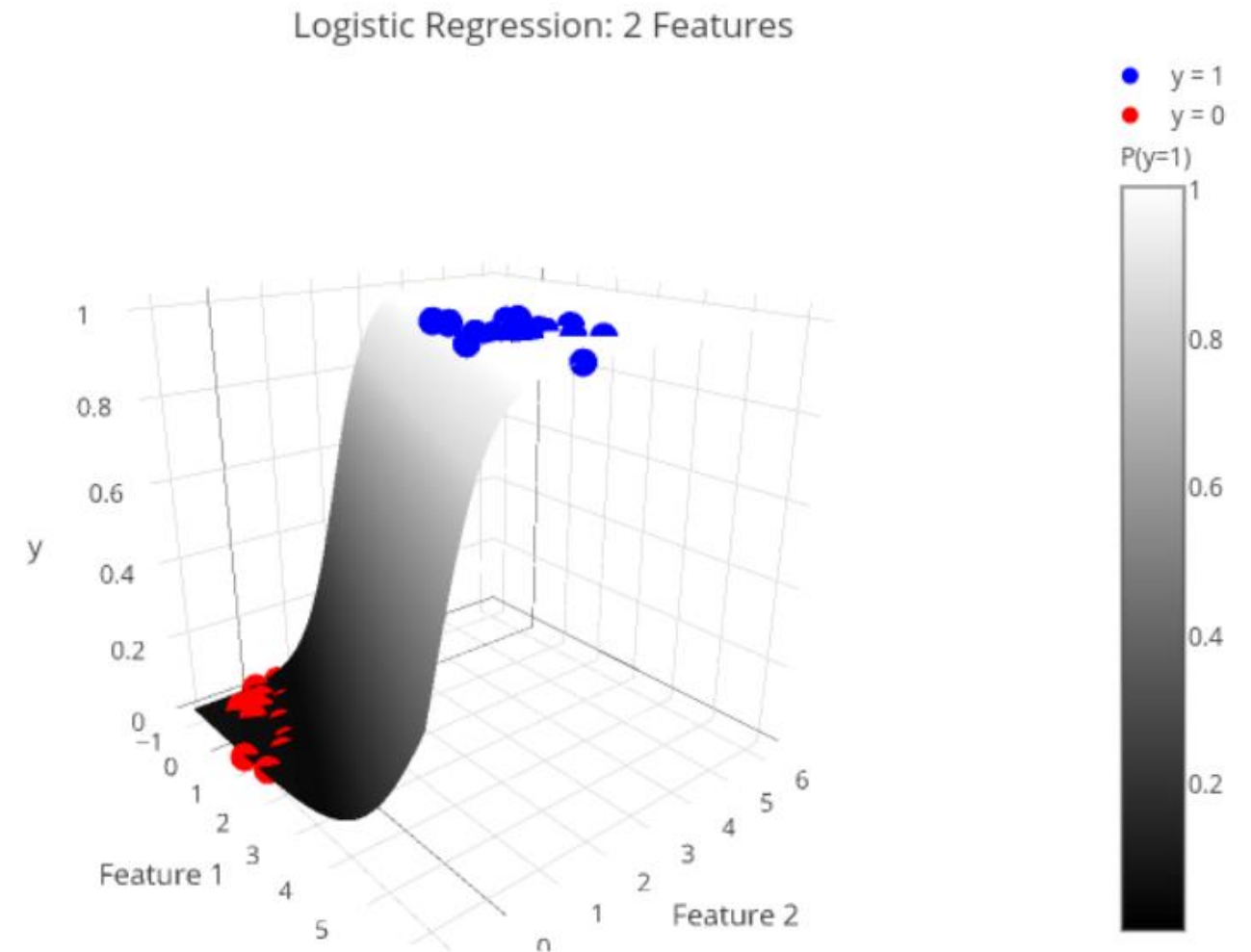
Logistische Regression



Logistische Regression

- Für Multiple Lineare Regression analog

$$\begin{aligned} Pr(Y = 1|\mathbf{X}) &= \sigma(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p) \\ &= \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}} \end{aligned}$$



Logistische Regression

- Training: Finde Gewichte, $\beta_0, \dots, \beta_p$ die zu den Trainingsdaten passen
- Labels $y_i \in \{0,1\}$, Vorhersagen müssen in $[0,1]$ liegen

- Vorhersagefunktion für Instanz i :

$$\hat{p}_i = \Pr(Y = 1 \mid X^{(i)}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)})}}$$

- Die Gewichte werden so gewählt, dass die Log-Likelihood des Modells maximiert wird

$$\ell(\beta_0, \beta_1, \dots, \beta_p) = \sum_{i=1}^n \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

- Die Log-Likelihood sagt: Wie wahrscheinlich sind die beobachteten Labels, wenn wir dieses Modell verwenden?
- Simple Methode:
 - Löse iterativ eine Folge von gewichteten least-square Regressionsproblemen, bis die log-likelihood in einem Maximum konvergiert
 - In der Praxis konvergiert das Verfahren meist nach wenigen Iterationen

Logistische Regression

- Multiclass Klassifikation
 - Ein Klassifikator wird für jedes Paar von Klassen gebaut (One-vs-One)
 - Nur Instanzen dieser Klassen werden benutzt
 - Ergebnis für einen unbekannten Datensatz basiert auf der Klasse, die am besten passt (Voting)
- Bringt gute Ergebnisse unter Beachtung des Klassifikationsfehlers
- Kann man auch zur Berechnung von Wahrscheinlichkeiten nutzen:
 - Paarweise koppeln
 - Kalibriert die individuellen Wahrscheinlichkeiten der verschiedenen Klassifikatoren

Logistische Regression

- Paarweise Klassifikation

- Für k Klassen werden $\frac{k(k-1)}{2}$ Klassifikatoren erstellt
- Ist dennoch genauso schnell wie die anderen Multiklassenmethoden
- Nur die Instanzen, die zu den zwei Klassen gehören, werden benutzt

- Gleichmäßige Verteilung von n Instanzen über k Klassen: $\frac{2n}{k}$ Instanzen pro Problem

- Laufzeit ist proportional zu

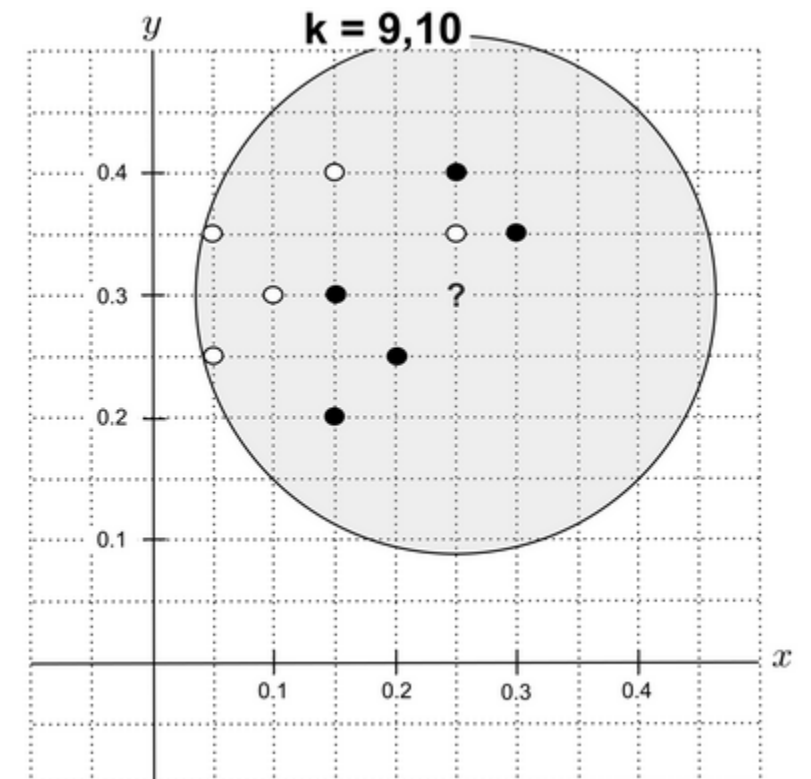
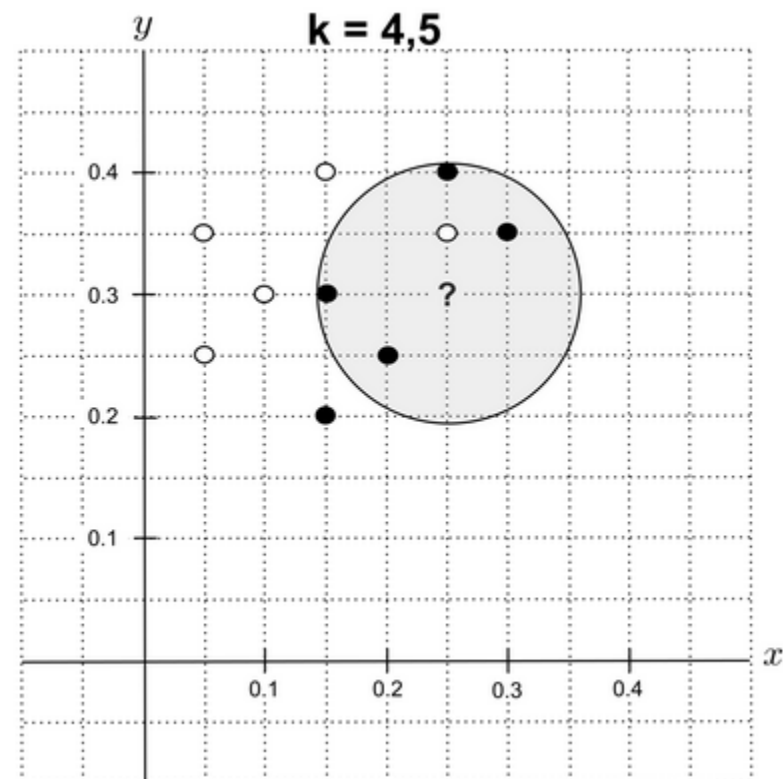
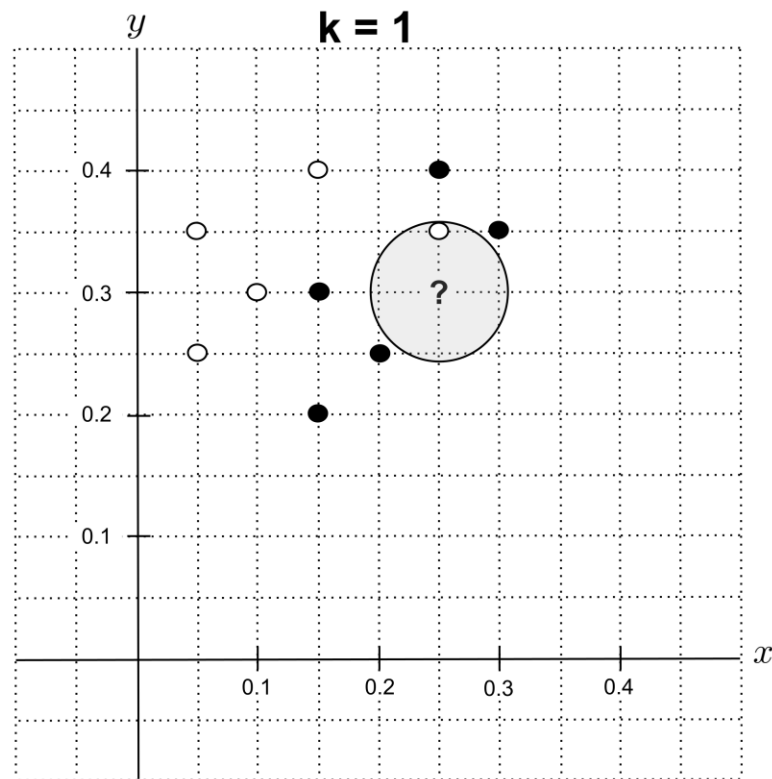
$$\frac{k(k-1)}{2} \cdot \frac{2n}{k} = (k-1) \cdot n$$

- Methode skaliert linear mit der Anzahl der Klassen

Nearest Neighbor

- Klassifiziere Datenpunkte aufgrund ihrer Nachbarschaft
- Attribute werden als Vektoren aufgefasst und in einen Raum gelegt
- Mehrheitsentscheidung der Trainingsdaten in der Nachbarschaft
- Algorithmus kann k Nachbarn in die Klassifikation einbeziehen
 - Zu wenig/ zu viele Nachbarn verändern die Ergebnisse
 - Zu klein (Overfitting), zu groß (Underfitting)
- Vektoren werden durch Distanzmaße auf „Nähe“/ „Ähnlichkeit“ geprüft
- Probleme:
 - Rechenzeit
 - Speicherbedarf
 - Verteilung der Daten im Raum

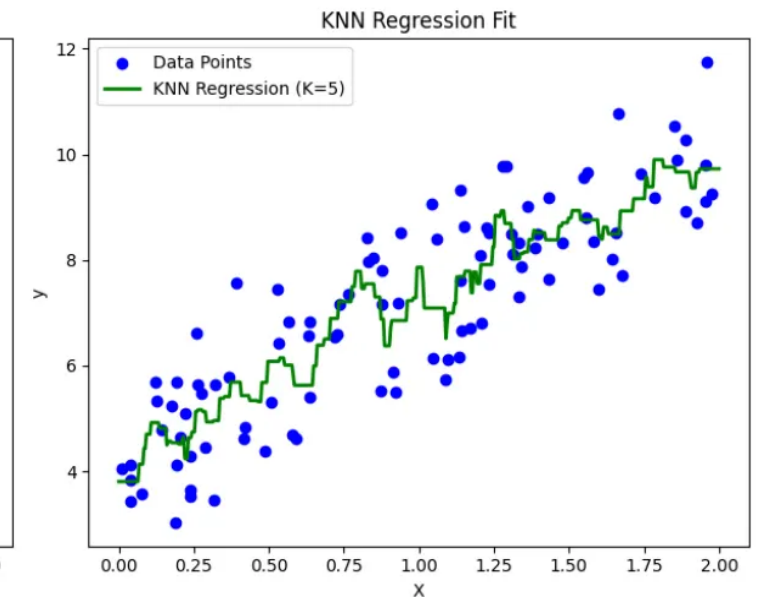
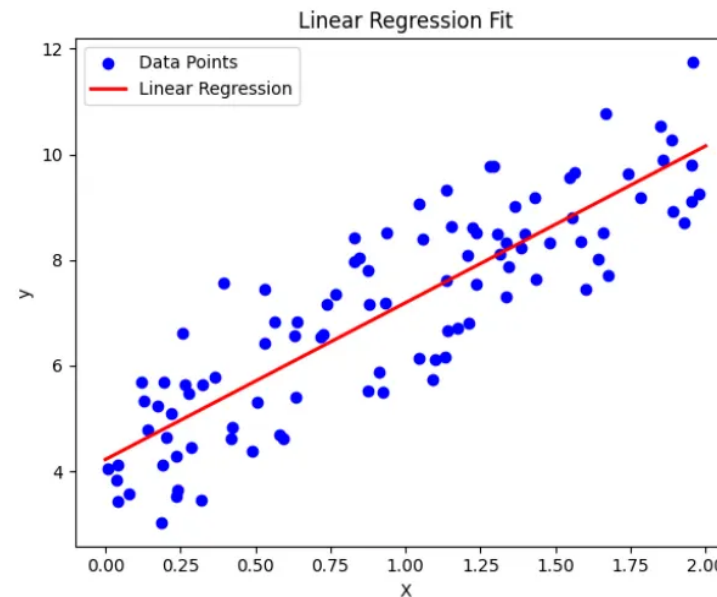
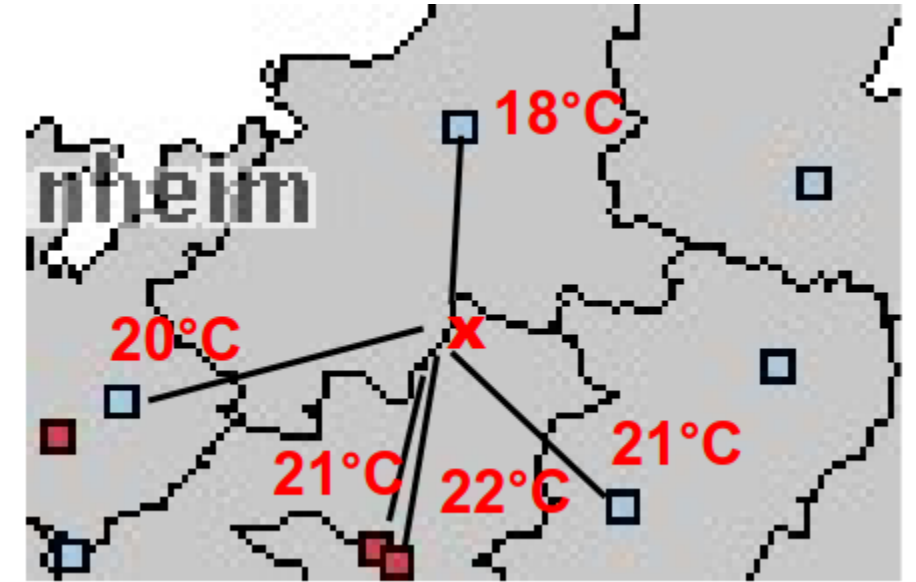
Nearest Neighbor



Bildquelle und Beispiel: <https://www.justinmath.com/k-nearest-neighbors/>

Nearest Neighbor

- Auch für Regression möglich
- Bildet Mittelwert über k Nachbarn
- Kann komplexe, nichtlineare Zusammenhänge abbilden
- Langsam bei großen Datensätzen:
Muss für jede Vorhersage viele Distanzen berechnen



Nearest Neighbor

- Damit man sagen kann, welche Instanzen „nah“ beieinander sind, braucht man eine Ähnlichkeitsfunktion $s(i,k)$.
- $s(i,k)$ gibt an, wie ähnlich Instanz i und Instanz k sind.
- Eigenschaften:
 - Symmetrisch: $s(i,k)=s(k,i)$
 - Nicht negativ: $s(i,k)\geq 0$
 - Selbstähnlichkeit: $s(i,i)=1$ (eine Instanz ist maximal ähnlich zu sich selbst)
- Man kann alle Werte in einer Ähnlichkeitsmatrix speichern
- Alternativ kann man auch ein Abstandsfunktion $t(i,k)$ verwenden, die jedem Paar (i,k) einen Wert zuordnet, der die „Distanz“ zwischen ihnen beschreibt
 - Kleinere Werte $\rightarrow$ Instanzen sind ähnlich
 - Größere Werte $\rightarrow$ Instanzen sind unähnlich
- Eigenschaften:
 - Symmetrisch: $t(i,k)=t(k,i)$
 - Nicht negativ: $t(i,k)\geq 0$
 - Identität: $t(i,i)=0$
- Man kann alle Werte in einer Abstandsmatrix speichern

Nearest Neighbor

- Minkowski Metriken (L_p Norm):

$$t_p(d_i, d_k) := \left(\sum_{j=1}^m |a_{ij} - a_{kj}|^p \right)^{1/p}$$

- Meist: Euklid'scher Abstand

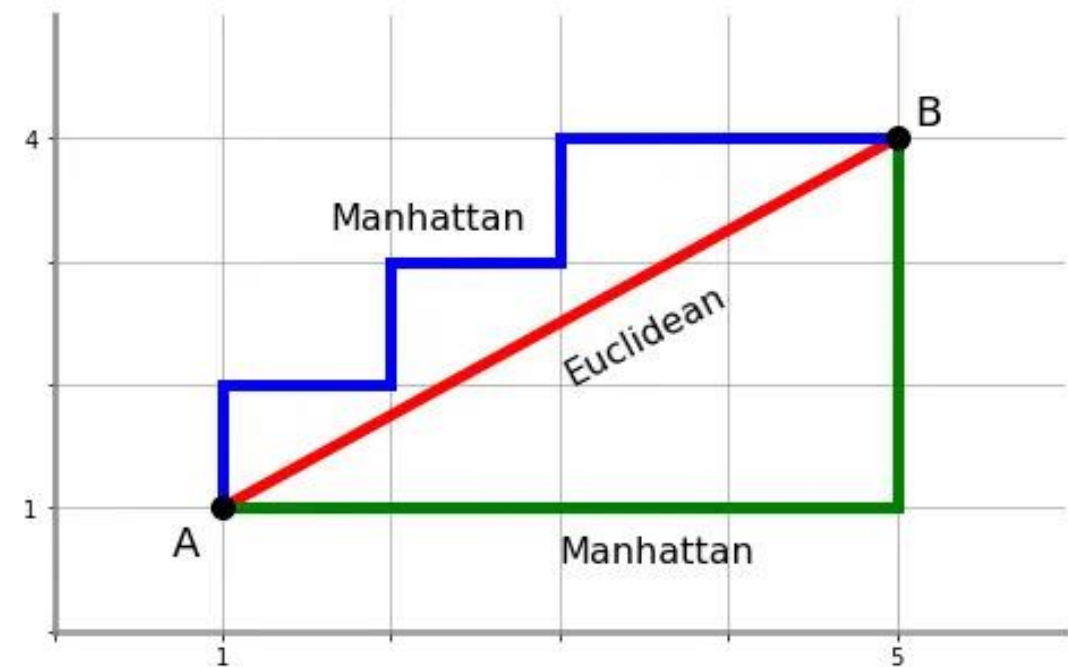
$$\sqrt{\sum_{j=1}^m (a_{ij} - a_{kj})^2}$$

- Alternative: Manhattan (city-block) Metrik

$$\sum_{k=1}^m |a_{ij} - a_{kj}|$$

- Bemerkungen

- Für Vergleiche kann die Wurzel weggelassen werden
- Invariant bezüglich Verschiebungen
- Nicht invariant bezüglich Skalierung



Nearest Neighbor

- Mahalanobis Abstand:

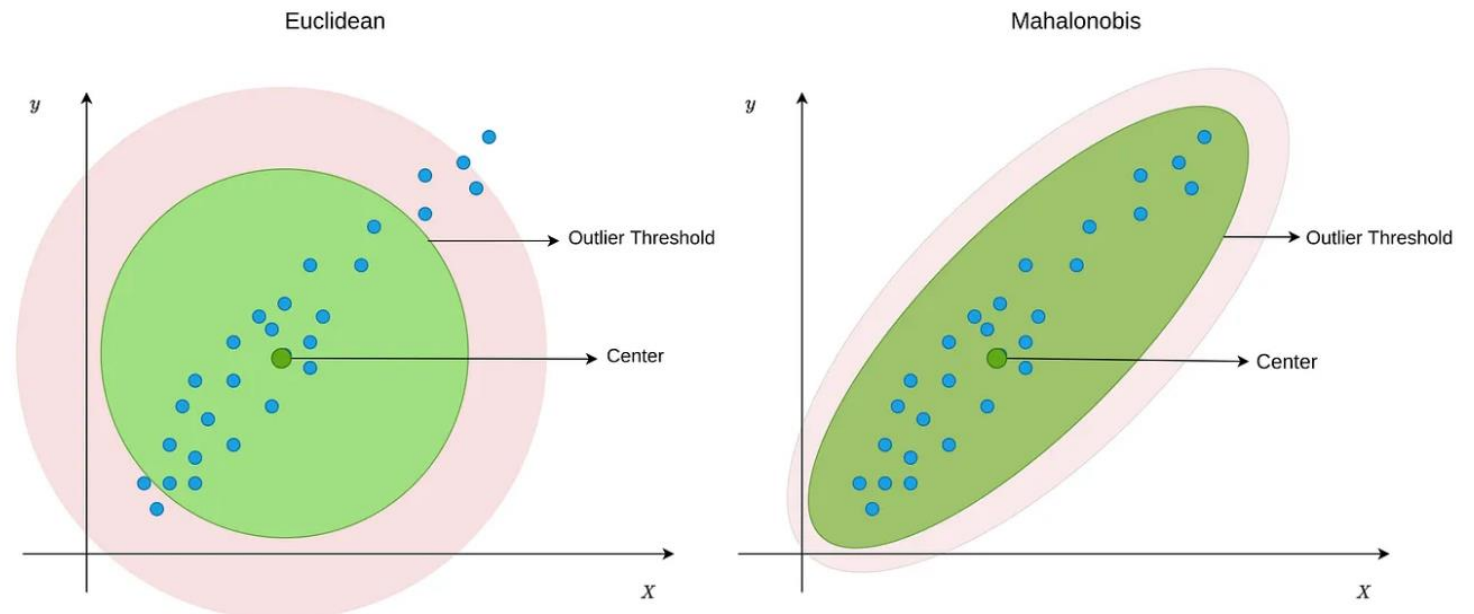
$$t_M(d_i, d_k) := (d_i - d_k)^T M^{-1} (d_i - d_k)$$

$$M := \frac{1}{n} \cdot \sum_{i=1}^n (d_i - \bar{d})(d_i - \bar{d})^T$$

$$\bar{d} := \frac{1}{n} \cdot \sum_{i=1}^n d_i$$

- Bemerkungen

- M : Co-Varianz Matrix
- Gibt Abstand in Standardabweichungen an
- Invariant bezüglich Verschiebungen
- Invariant bezüglich Skalierung



Nearest Neighbor

Algorithmus Aufbau k-d-Baum

– Initialisierung

1. Sortiere Punkte für jedes Attribut
2. $i \leftarrow 1$
3. Erzeuge Knoten
4. Starte Rekursion

– Rekursion

1. Teile alle Datenpunkte bezüglich Attribut i in zwei gleich große Teilmengen (verwende Median)
2. Trage Unterteilungswert t in aktuellen Knoten ein

3. Erzeuge linken Kindknoten

1. Weise ihm seine Datenpunkte zu
2. Falls Anzahl der Datenpunkte größer als Schwellwert
 1. $i \leftarrow i + 1$
 2. Rekursiver Aufruf

4. Erzeuge rechten Kindknoten

1. Weise ihm seine Datenpunkte zu
2. Falls Anzahl der Datenpunkte größer als Schwellwert
 1. $i \leftarrow i + 1$
 2. Rekursiver Aufruf

Nearest Neighbor

Algorithmus Aufbau k-d-Baum

– Initialisierung

1. Sortiere Punkte für jedes Attribut
2. $i \leftarrow 1$
3. Erzeuge Knoten
4. Starte Rekursion

– Rekursion

1. Teile alle Datenpunkte bezüglich Attribut i in zwei gleich große Teilmengen (verwende Median)
2. Trage Unterteilungswert t in aktuellen Knoten ein

Tafel $\rightarrow$ Punkte = [A = (3, 6), B = (17, 15), C = (13, 15), D = (6, 12), E = (9, 1), F = (2, 7), G = (10, 19)]

3. Erzeuge linken Kindknoten

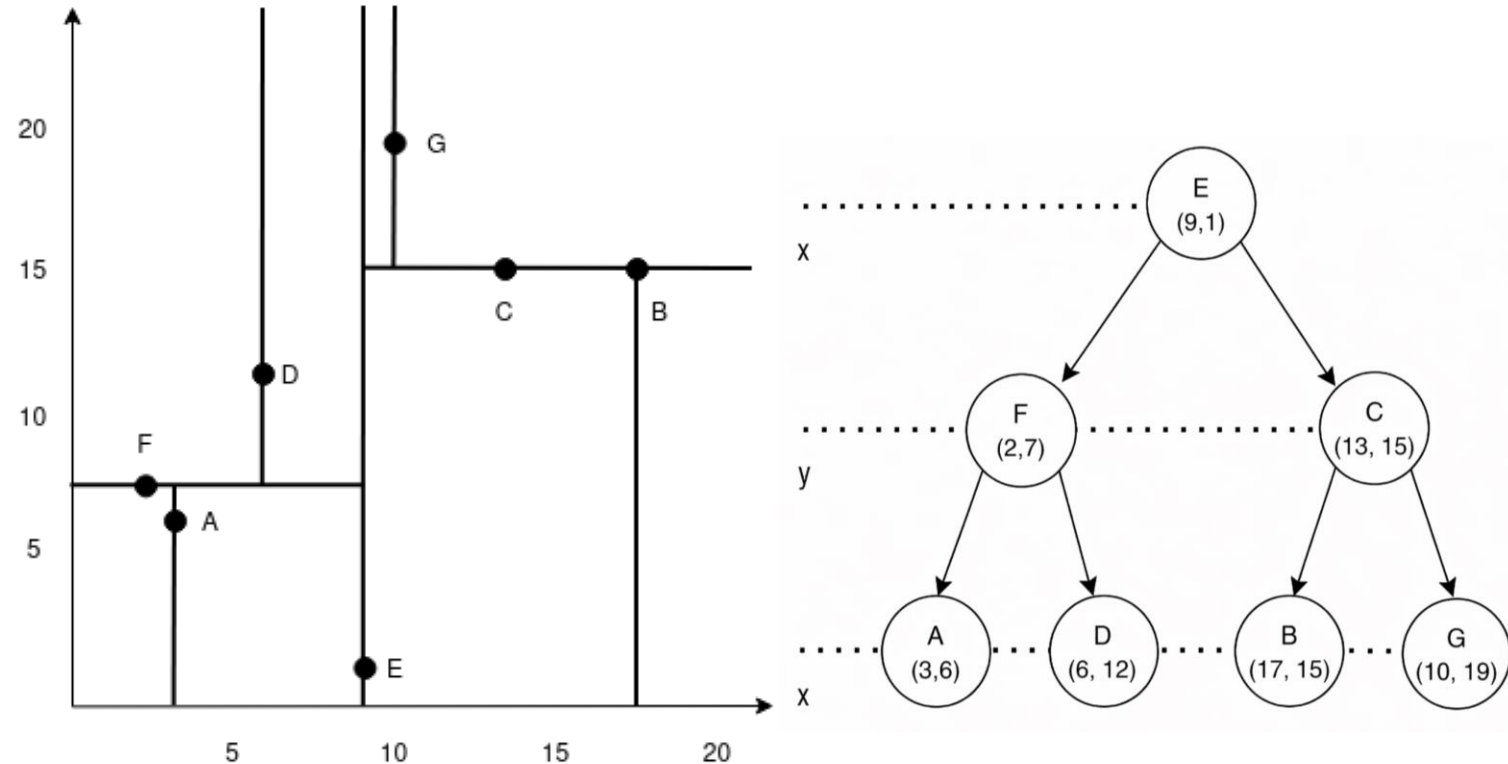
1. Weise ihm seine Datenpunkte zu
2. Falls Anzahl der Datenpunkte größer als Schwellwert
 1. $i \leftarrow i + 1$
 2. Rekursiver Aufruf

4. Erzeuge rechten Kindknoten

1. Weise ihm seine Datenpunkte zu
2. Falls Anzahl der Datenpunkte größer als Schwellwert
 1. $i \leftarrow i + 1$
 2. Rekursiver Aufruf

k-d-Baum

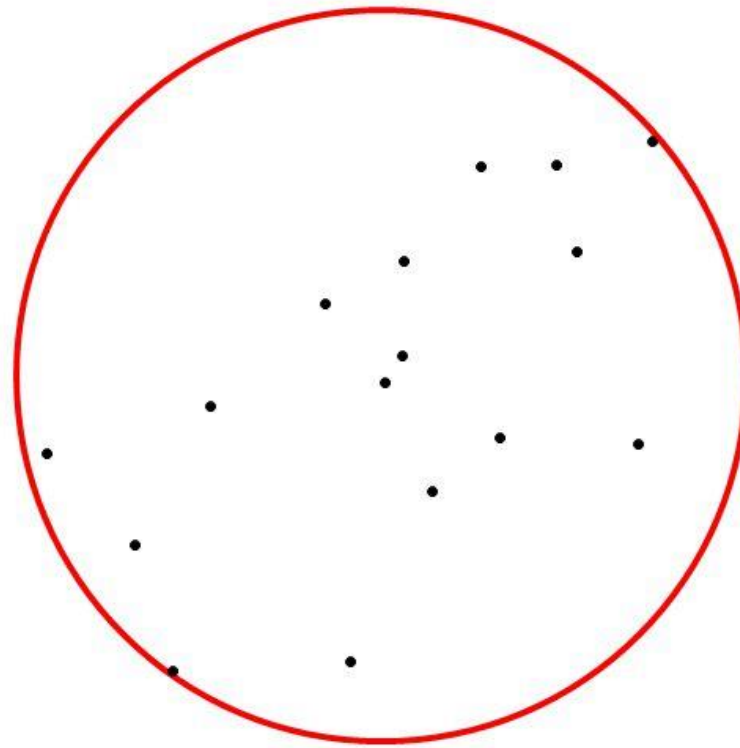
- Punkte = [A = (3, 6), B = (17, 15), C = (13, 15), D = (6, 12), E = (9, 1), F = (2, 7), G = (10, 19)]
- Ebene 1 (x-Achse): Wähle Median in x:
 - Sortiere x-Werte: 2,3,6,9,10,13,17
→ Median = 9 → Wurzel = E(9,1)
- Ebene 2 (y-Achse):
 - Linke Unterbäume ($x < 9$): F(2,7), A(3,6), D(6,12) → Median y = 7 → Knoten = F(2,7)
 - Rechte Unterbäume ($x > 9$): B(17,15), C(13,15), G(10,19) → Median y = 15 → Knoten = C(13,15)
- Rekursiv fortsetzen → Blätter: A(3,6), D(6,12), B(17,15), G(10,19)



Ball Tree

- k-d-Bäume sind aufgrund ihrer Ecken nicht ideal
 - Gehen davon aus, dass die Daten relativ gleichmäßig verteilt sind entlang jeder Dimension
 - Das ist oft nicht der Fall
- Nutzung von Sphären:
 - Zerlege den Raum in rekursiv kleiner werdende überlappende Sphären
- Aufbau:
 - Wähle einen Punkt möglichst entfernt vom Mittelpunkt
 - Wähle einen zweiten möglichst weit entfernt vom ersten
 - Ordne alle Punkte dem nächsten der beiden Punkte zu
 - Berechne den Zentroid der beiden Mengen
 - Nutze den Zentroid als Sphärenmittelpunkt und die Distanz zum entferntesten Punkt als Radius
 - Zerlege die Teilmenge rekursiv weiter

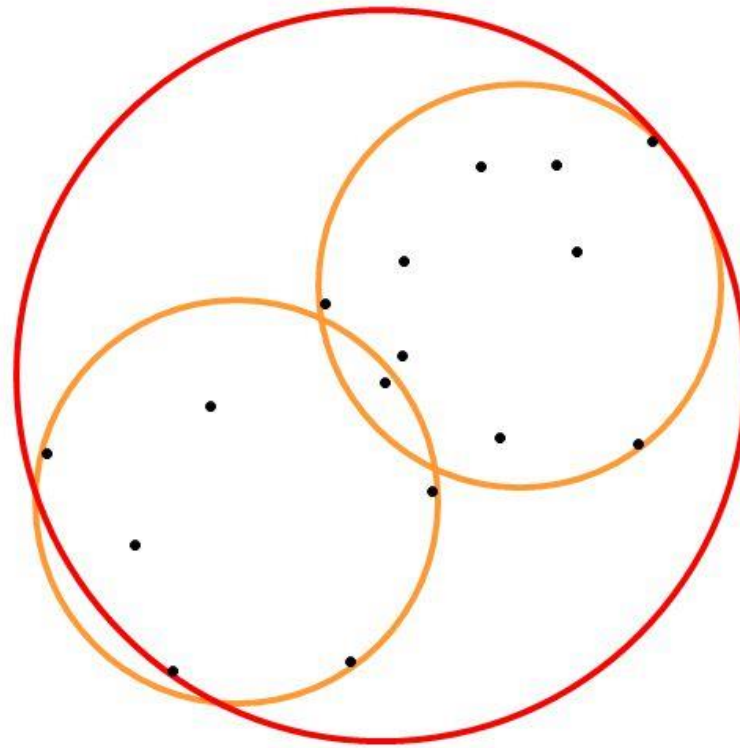
Ball Tree



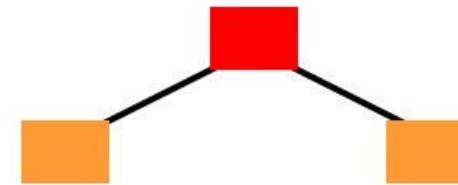
A ball-tree: level 1



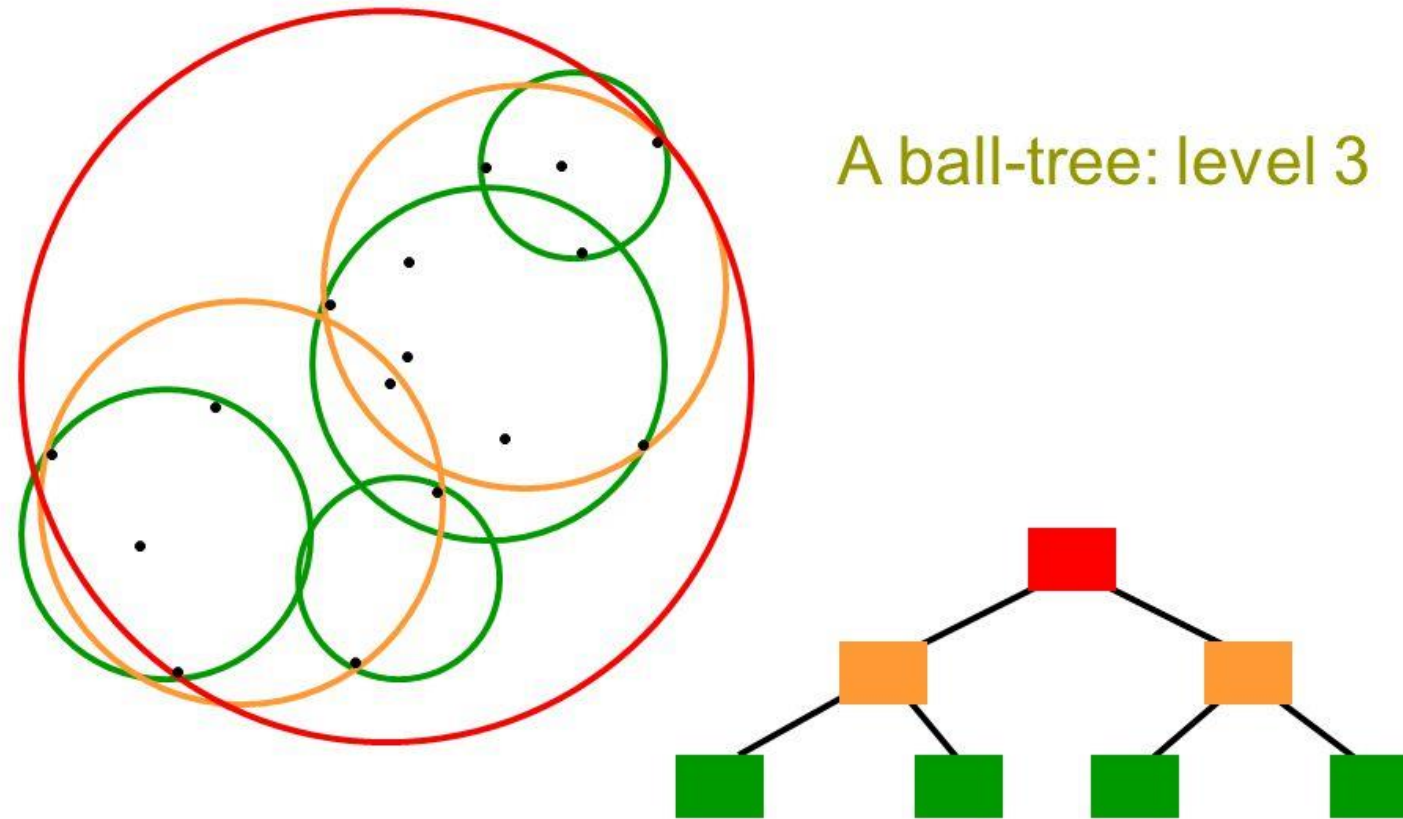
Ball Tree



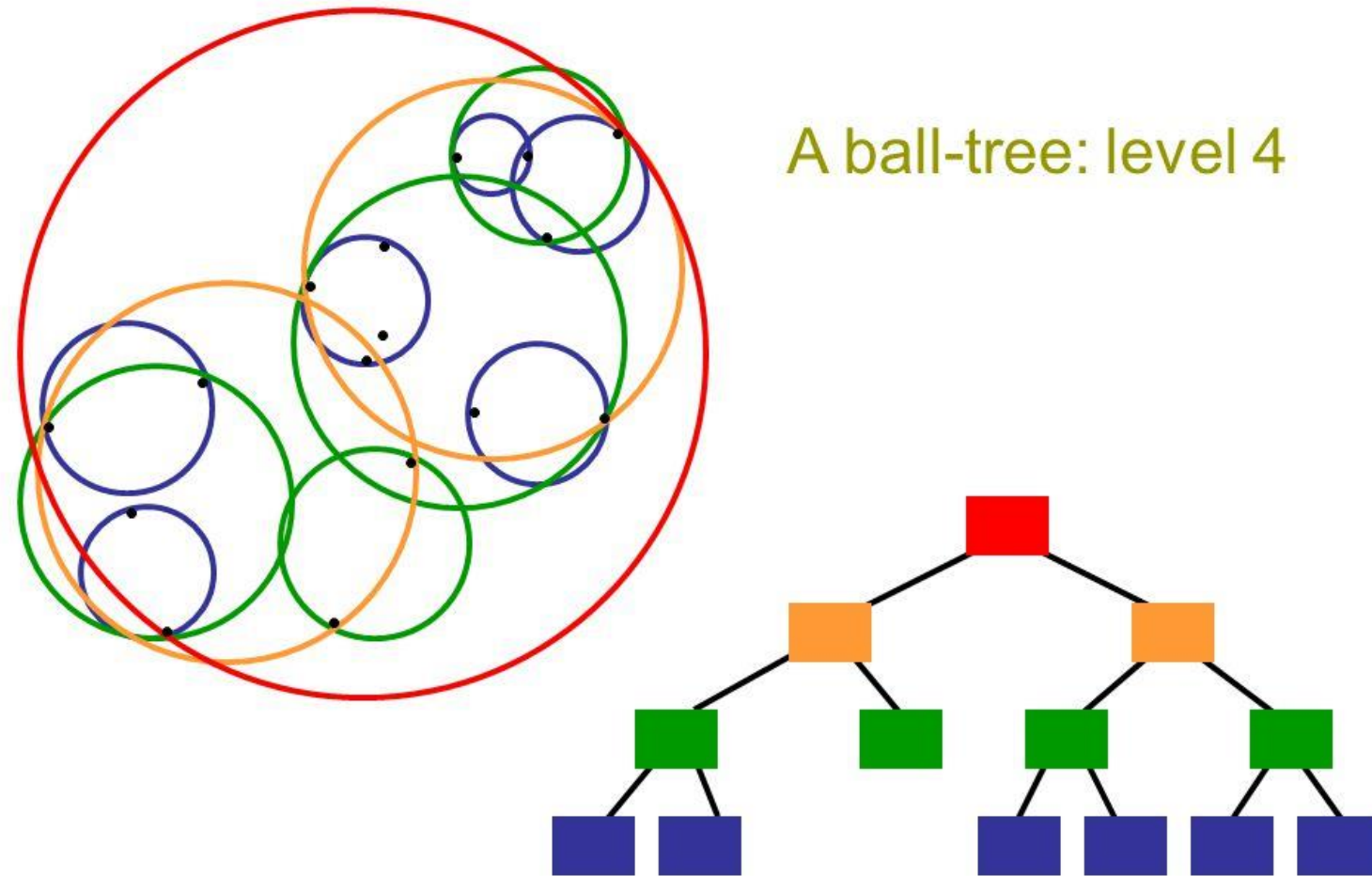
A ball-tree: level 2



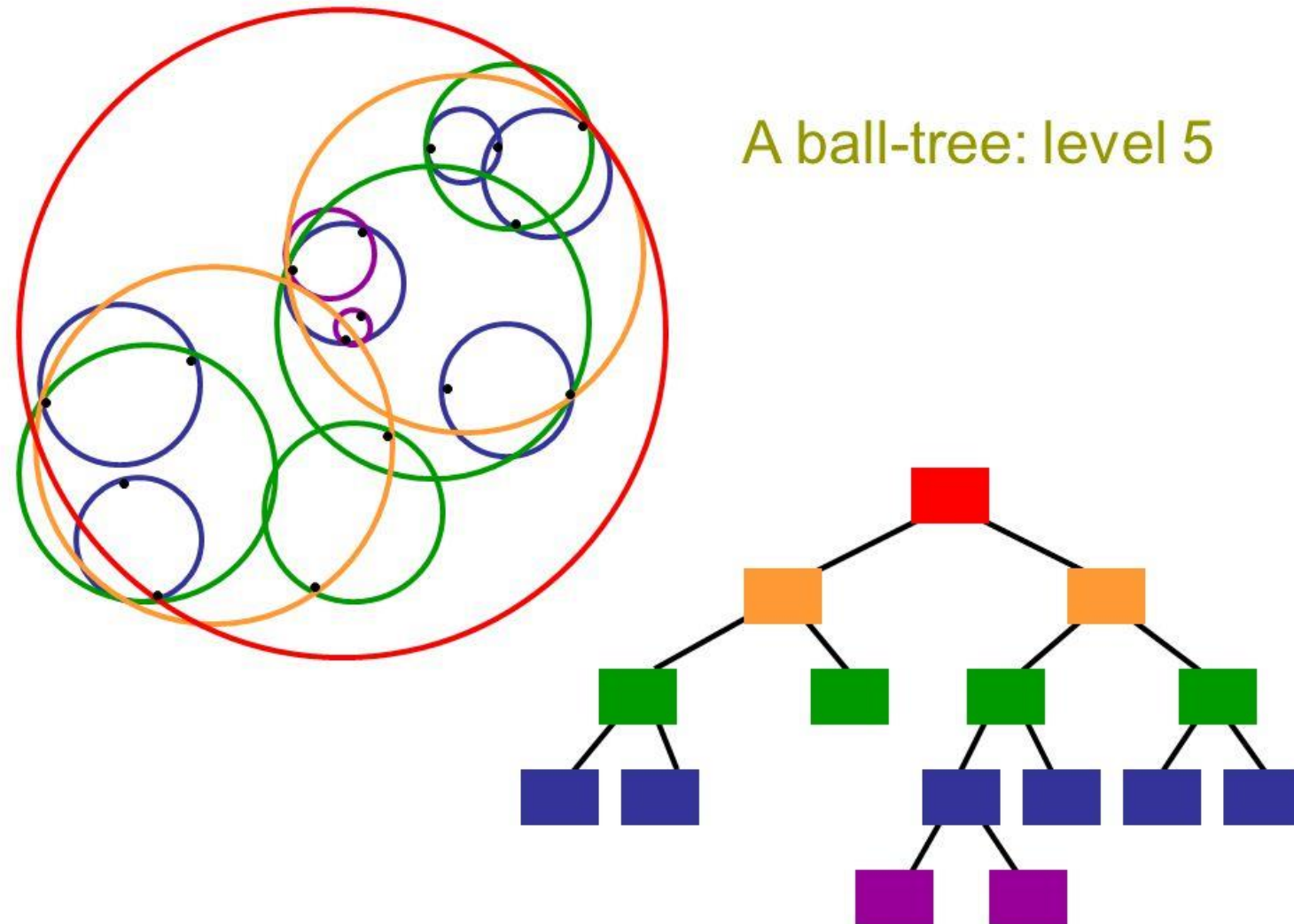
Ball Tree



Ball Tree



Ball Tree



Weitere Verfahren

Baumtyp	Struktur	Dimension	Updates
k-d Tree	Jeder Knoten teilt den Raum rekursiv entlang einer Achse	Niedrig ($<10-20$)	Schwierig, da Splits starr sind
Ball Tree	Jeder Knoten speichert einen Mittelpunkt und Radius; Punkte im Ball werden rekursiv in Unterbälle aufgeteilt	Niedrig-mittel ($\sim 10-50$)	Teilweise aufwendig, da Mittelpunkt und Radius neu berechnet werden müssen
Cover Tree	Nicht-binär; hierarchische Levels mit abnehmendem Radius. Höhere Ebenen decken tiefere ab, Punkte auf gleicher Ebene haben Mindestabstand	Mittel-hoch ($\sim 50-200$)	Dynamisch und effizienter, keine vollständige Rekonstruktion nötig, nur Zeiger-Updates

- Bei größeren Dimensionen (ca. ab 50-100) und riesigen Datensätzen:
Approximiertes k-NN
 - HNSW – Hierarchical Navigable Small World Graphs (Ungenauigkeit durch Greedy Suche auf Graphen)
 - IVF – Inverted File (Ungenauigkeit durch Suche in Clustern)